

Project: Interactive To-Do List (Vanilla JS + DOM + LocalStorage)

Project Scenario

Tum ek productivity app startup ke frontend developer ho — bina kisi framework ke (pure vanilla JS), ek production-quality To-Do List banana hai.

Features: Add/Delete/Toggle tasks, filter (All/Active/Done), localStorage persistence, event delegation (efficient), aur XSS-safe rendering.

Ye exact pattern hai jo React jaisi libraries internally (DOM diffing se pehle) karte the!

Step 1 — HTML Structure

index.html

```
<!DOCTYPE html>
<html lang='en'>
<head>
<meta charset='UTF-8'>
<title>To-Do List</title>
<link rel='stylesheet' href='style.css'>
</head>
<body>
<div class='app'>
<h1>My Tasks</h1>
<form id='todo-form'>
<input type='text' id='todo-input' placeholder='Add a new task...' required>
<button type='submit'>Add</button>
</form>
<div class='filters'>
<button class='filter-btn active' data-filter='all'>All</button>
<button class='filter-btn' data-filter='active'>Active</button>
<button class='filter-btn' data-filter='done'>Done</button>
</div>
<ul id='todo-list'></ul>
<div class='footer'>
<span id='item-count'>0 items left</span>
<button id='clear-completed'>Clear Completed</button>
</div>
</div>
<script src='app.js' type='module'></script>
</body>
```

Step 2 — State Management Module (state.js)

Closure pattern (Day 2 concept!) se private state — localStorage sync ke saath.

state.js — Closure-based Store

```
// state.js
const STORAGE_KEY = 'todos-v1';

function createStore() {
  let todos = load();
  let filter = 'all';
  const listeners = [];

  function load() {
    try {
      const raw = localStorage.getItem(STORAGE_KEY);
      return raw ? JSON.parse(raw) : [];
    } catch {
      return [];
    }
  }

  function persist() {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(todos));
  }

  function notify() { listeners.forEach(fn => fn(getVisibleTodos())); }

  function getVisibleTodos() {
    if (filter === 'active') return todos.filter(t => !t.done);
    if (filter === 'done') return todos.filter(t => t.done);
    return [...todos];
  }

  return {
    subscribe(fn) { listeners.push(fn); fn(getVisibleTodos()); },
    addTodo(text) {
      todos.push({ id: Date.now(), text, done: false });
      persist(); notify();
    },
    deleteTodo(id) {
      todos = todos.filter(t => t.id !== id);
      persist(); notify();
    },
    toggleTodo(id) {
      todos = todos.map(t => t.id === id ? {...t, done: !t.done} : t);
      persist(); notify();
    },
    clearCompleted() {
      todos = todos.filter(t => !t.done);
      persist(); notify();
    },
    setFilter(f) { filter = f; notify(); },
  };
}
```

```

getStats() {
  return {
    total: todos.length,
    active: todos.filter(t => !t.done).length,
    done: todos.filter(t => t.done).length,
  };
},
getFilter() { return filter; },
};
}

export const store = createStore();

```

Step 3 — Rendering Module (render.js)

XSS-safe DOM rendering — `textContent` everywhere for user data.

render.js — Safe DOM Rendering

```

// render.js
export function renderTodos(todos, listEl) {
  listEl.innerHTML = ''; // safe – clearing, not inserting user data
  if (todos.length === 0) {
    const empty = document.createElement('li');
    empty.className = 'empty-state';
    empty.textContent = 'No tasks here. Add one above!';
    listEl.append(empty);
    return;
  }
  todos.forEach(todo => {
    const li = document.createElement('li');
    li.className = 'todo-item' + (todo.done ? ' completed' : '');
    li.dataset.id = todo.id;
    const checkbox = document.createElement('input');
    checkbox.type = 'checkbox';
    checkbox.checked = todo.done;
    checkbox.className = 'toggle';
    const span = document.createElement('span');
    span.textContent = todo.text; // ■ SAFE – XSS-proof
    span.className = 'todo-text';
    const delBtn = document.createElement('button');
    delBtn.textContent = '\u2715'; // ✕
    delBtn.className = 'delete-btn';
    delBtn.setAttribute('aria-label', 'Delete task');
    li.append(checkbox, span, delBtn);
    listEl.append(li);
  });
}

export function renderStats(stats, countEl) {
  countEl.textContent = `${stats.active} item${stats.active !== 1 ? 's' : ''} left`;
}

```

```

}

export function renderFilters(activeFilter, filterBtns) {
  filterBtns.forEach(btn => {
    btn.classList.toggle('active', btn.dataset.filter === activeFilter);
  });
}

```

Step 4 — Main App (app.js) — Event Delegation Wiring

app.js — Wiring Everything Together

```
// app.js
import { store } from './state.js';
import { renderTodos, renderStats, renderFilters } from './render.js';

const form = document.getElementById('todo-form');
const input = document.getElementById('todo-input');
const list = document.getElementById('todo-list');
const countEl = document.getElementById('item-count');
const clearBtn= document.getElementById('clear-completed');
const filterBtns = document.querySelectorAll('.filter-btn');

// ■■ Add new todo ████████████████████████████████████████████████████████████
form.addEventListener('submit', (e) => {
e.preventDefault(); // stop page reload!

const text = input.value.trim();
if (!text) return;
store.addToDo(text);
input.value = '';
input.focus();
});

// ■■ Event Delegation: toggle + delete (single listener!) ██████████
list.addEventListener('click', (e) => {
const li = e.target.closest('.todo-item');
if (!li) return;
const id = Number(li.dataset.id);
if (e.target.matches('.toggle')) {
store.toggleToDo(id);
} else if (e.target.matches('.delete-btn')) {
li.classList.add('removing'); // CSS animation hook
setTimeout(() => store.deleteToDo(id), 200);
}
});

// ■■ Filter buttons (event delegation on parent) ████████████████████████████████████████
document.querySelector('.filters').addEventListener('click', (e) => {
if (!e.target.matches('.filter-btn')) return;
store.setFilter(e.target.dataset.filter);
});

// ■■ Clear completed ████████████████████████████████████████████████████████████████████████
clearBtn.addEventListener('click', () => store.clearCompleted());

// ■■ Subscribe to store changes – re-render on EVERY change ████
```

```
store.subscribe((visibleTodos) => {  
  renderTodos(visibleTodos, list);  
  renderStats(store.getStats(), countEl);  
  renderFilters(store.getFilter(), filterBtns);  
});
```

■ Architecture Notes — Why This Structure?

state.js — single source of truth (sounds like Redux? It IS the same idea!).

render.js — pure rendering functions, no state mutation, XSS-safe.

app.js — only event wiring, calls store methods, subscribes to changes.

store.subscribe() pattern — observer pattern, foundation for React/Vue reactivity!

Bonus Challenges

1. Edit-in-place: double-click todo text -> turns into input field -> Enter saves.
2. Drag-and-drop reordering using HTML5 Drag & Drop API.
3. Due dates: add date picker, show 'overdue' badge with red color.
4. Keyboard shortcuts: 'a' to focus input, Escape to clear input.
5. Sync across tabs using the 'storage' event (see Theory Topic 5).
6. Categories/tags — filter by tag in addition to status.